Universidad de los Andes
Colombia
Departamento de Ingeniería de Sistemas y Computación

ArchIA: Herramienta basada en inteligencia artificial generativa para la asistencia en la formalización de drivers arquitectónicos siguiendo Attribute-Driven Design 3.0

Santiago Casasbuenas
Juan Pablo Castro
Manuel Gomez

Director: Dario Correal

Bogotá, Colombia
2026

Contents

1 Introducción **1**

2 Agradecimientos **1**

3 Marco Teórico **1**

3.1 Inteligencia Artificial Generativa (IAG) 1
3.2 Modelos de Lenguaje de Gran Escala (LLM) 1
3.3 Sistemas Multi-Agente 1
3.4 Recuperación Aumentada con Generación (RAG) 1
3.5 Memoria Persistente en Sistemas Conversacionales 1
3.6 Arquitectura de Software y Attribute-Driven Design 3.0 (ADD 3.0) 1

4 Problema **1**

5 Objetivos **2**

5.1 Objetivo General 2
5.2 Objetivos Específicos 2

6 Antecedentes **2**

7 Definición de Problemática y Alcance del Proyecto **2**

8 Limitaciones del Proyecto **2**

9 Diseño e Implementación **2**

9.1 Arquitectura General del Sistema 2
9.2 Backend 2
9.2.1 Modularización del código 3
9.2.2 Sistema de RAG indexado por atributo de calidad 3
9.2.3 Optimizaciones de rendimiento 5
9.3 Frontend 7
9.4 Diseño Anterior del Grafo 7
9.4.1 Flujo original del sistema 8
9.4.2 Diagnóstico: fortalezas y debilidades 8
9.4.3 Problemas críticos de diseño 8
9.4.4 Feedback del asesor que motivó el rediseño 9
9.5 Diseño Final del Grafo 9

9.5.1 Flujo actual del grafo 10
9.5.2 Nodos especializados 10
9.5.3 Soporte multi-intent 12
9.5.4 Enrutamiento dinámico por atributo de calidad 12
9.5.5 Design Ledger: trazabilidad de decisiones 13
9.5.6 ASR Write-back y Supersession 14

10 Descripción de la Herramienta 14

11 Resultados 14

12 Pruebas 14

13 Conclusiones 14

14 Trabajo Futuro 14

A Anexo A: System Prompts Principales 17

B Anexo B: Ejemplo Completo de Ejecución 17

C Anexo C: Documentación Técnica Complementaria 17

List of Figures

List of Tables

1 Distribución de chunks en el vectorstore por tipo de contenido 5
2 Cuellos de botella identificados y estado de resolución 5
3 Diagnóstico del estado inicial de ARCHIA 8

1 Introducción

[TODO: Escribir introducción — ver guía arriba]

2 Agradecimientos

[TODO: Escribir agradecimientos personales]

3 Marco Teórico

3.1 Inteligencia Artificial Generativa (IAG)

[TODO: Escribir subsección IAG]

3.2 Modelos de Lenguaje de Gran Escala (LLM)

[TODO: Escribir subsección LLM]

3.3 Sistemas Multi-Agente

[TODO: Escribir subsección multi-agente]

3.4 Recuperación Aumentada con Generación (RAG)

[TODO: Escribir subsección RAG]

3.5 Memoria Persistente en Sistemas Conversacionales

[TODO: Escribir subsección memoria]

3.6 Arquitectura de Software y Attribute-Driven Design 3.0 (ADD 3.0)

[TODO: Escribir subsección ADD 3.0]

4 Problema

[TODO: Escribir planteamiento del problema]

5 Objetivos

5.1 Objetivo General

[TODO: Redactar objetivo general]

5.2 Objetivos Específicos

[TODO: Redactar objetivos específicos]

6 Antecedentes

[TODO: Escribir antecedentes y estado del arte]

7 Definición de Problemática y Alcance del Proyecto

[TODO: Escribir definición de alcance]

8 Limitaciones del Proyecto

[TODO: Escribir limitaciones]

9 Diseño e Implementación

9.1 Arquitectura General del Sistema

[TODO: Escribir arquitectura general]

9.2 Backend

El backend de ARCHIA está construido con FastAPI sobre Python y constituye el núcleo del sistema. Su responsabilidad principal es recibir los mensajes del usuario, orquestar la ejecución de los agentes especializados mediante LangGraph y devolver respuestas estructuradas al frontend. Esta subsección describe las tres dimensiones más relevantes: la modularización del código, el sistema de RAG indexado y las optimizaciones de rendimiento.

9.2.1 Modularización del código

El archivo monolítico `graph.py` de la versión inicial se descompuso en un paquete con responsabilidades claramente separadas. El Listing 1 muestra la estructura resultante.

```
(* \texttt{back/src/graph/} *)
  __init__.py
  workflow.py      # Estructura del grafo (nodos, edges, compilacion)
  state.py         # GraphState (TypedDict) + schemas de validacion
  consts.py        # Prompts del sistema y esquemas de tacticas
  utils.py         # Funciones auxiliares
  resources.py     # Recursos compartidos (LLM, RAG, checkpointer)
  qa_registry.py   # Registro central de atributos de calidad
  index_resolver.py # Resolucion LLM del indice QA
  nodes/           # Directorio de nodos especializados
    supervisor.py    classifier.py    investigator.py
    asr.py           evaluator.py     unifier.py
    diagram.py       context_loader.py
    styles/          # Nodos de estilo por atributo de calidad
    tactics/         # Nodos de tacticas por atributo de calidad
```

Listing 1: Estructura del paquete `graph/` tras la modularización

Cada nodo es una función pura que recibe un `GraphState` y retorna un `GraphState` modificado. Esta separación permite probar cada nodo aisladamente y agregar nuevos nodos sin modificar el cableado existente: basta con registrar el nodo en `workflow.py` y conectar sus edges.

9.2.2 Sistema de RAG indexado por atributo de calidad

La versión inicial del RAG utilizaba una sola colección de ChromaDB sin diferenciación temática. Esto significaba que una pregunta sobre tácticas de escalabilidad y una sobre latencia obtenían el mismo pool de documentos, reduciendo la precisión de las recomendaciones.

Se implementó un sistema de RAG indexado con dos dimensiones de metadata por chunk: `quality_attribute` (escalabilidad, latencia, disponibilidad, general) y `content_type` (asr, estilos, tácticas, general). Estas etiquetas se asignan automáticamente durante la ingestión de PDFs mediante un algoritmo de auto-tagging basado en keywords.

El Listing 2 muestra el algoritmo de etiquetado implementado en `build_vectorstore.py`.

```python
# Archivo: back/build_vectorstore.py (lineas 44-70)
def _tag_chunk(text, config):
    tokens = text.lower().split()
    # Para cada QA en indices.json: contar matches de keywords
```

```
    qa_scores = {}
    for qa in config["quality_attributes"]:
        score = sum(1 for kw in qa["keywords_en"] + qa["keywords_es"]
                    if kw in tokens)
        qa_scores[qa["id"]] = score
    best_qa = max(qa_scores, key=qa_scores.get)
    quality_attribute = best_qa if qa_scores[best_qa] > 0 else "general"
    # Mismo proceso para content_type
    ...
    return {"quality_attribute": quality_attribute,
            "content_type": content_type}
```

Listing 2: Algoritmo de auto-tagging de chunks (simplificado)

La resolución del atributo de calidad en tiempo de ejecución se realiza mediante inferencia LLM, no por keywords. El nodo `classifier` invoca la función `resolve_quality_attribute()`, implementada en `graph/index_resolver.py` (líneas 28–74), que consulta al LLM con la lista de atributos disponibles cargada desde `config/indices.json`. Si el LLM no identifica un atributo específico, el sistema retorna `"general"` como fallback.

El retriever diferenciado se obtiene mediante `get_indexed_retriever()`, que aplica filtros de metadata sobre ChromaDB. Por ejemplo, para una solicitud de tácticas de escalabilidad:

```
# Archivo: back/src/graph/nodes/tactics/common.py (lineas 51-55)
_retriever = get_indexed_retriever(
    quality_attribute=normalize_qa(resolved_index or qa),
    content_type="tacticas",
    k=k,
)
```

Listing 3: Retriever indexado con filtros de metadata

El corpus RAG contiene 658 chunks etiquetados provenientes de *Software Architecture in Practice* y *Evaluating Software Architecture*. La Table 1 muestra la distribución tras la optimización de keywords.

Table 1: Distribución de chunks en el vectorstore por tipo de contenido

content_type	Chunks	% total
asr	391	59.4%
general	164	24.9%
tácticas	57	8.7%
estilos	46	7.0%
Total	**658**	**100%**

Para añadir un nuevo atributo de calidad al sistema (por ejemplo, seguridad), basta con editar `config/indices.json` y reconstruir el vectorstore. No se requieren cambios de código: el resolver LLM detecta automáticamente los nuevos índices gracias a la configuración declarativa.

9.2.3 Optimizaciones de rendimiento

Los tiempos de respuesta iniciales del sistema eran inaceptables para uso interactivo: hasta 350 segundos en prompts multi-intent. Se realizó un diagnóstico que identificó siete cuellos de botella, de los cuales se resolvieron cuatro durante el semestre. La Table 2 resume los problemas y su estado.

Table 2: Cuellos de botella identificados y estado de resolución

Prior.	Problema	Impacto	Dificultad	Estado
P1	Llamadas LLM secuenciales	60–70% de latencia total	Media	Pendiente
P2	Sin streaming al cliente	Latencia percibida alta	Baja	Pendiente
P3	RAG queries secuenciales	15–20% de latencia	Baja	Resuelto
P4	Sin caché	40–60% cómputo redundante	Media	Resuelto
P5	Graphviz bloqueante	5–15 s en diagramas	Baja	Resuelto
P6	Un solo worker Uvicorn	Concurrencia bloqueada	Muy baja	Pendiente
P7	State/prompt sin límite	Degradación progresiva	Baja	Pendiente

9.2.3.1 P3 — Paralelización de queries RAG. Las búsquedas vectoriales en ChromaDB eran completamente independientes entre sí, pero se ejecutaban en un bucle secuencial. Se reemplazó el bucle por un `ThreadPoolExecutor` que ejecuta todas las queries en paralelo. Se eligió `ThreadPoolExecutor` en lugar de `asyncio.gather` porque el método `invoke()` del retriever de LangChain es síncrono.

```
47 # Archivo: back/src/graph/nodes/styles/common.py (lineas 47-62)
48 with ThreadPoolExecutor(max_workers=len(queries)) as executor:
49     futures = {executor.submit(_retriever.invoke, q): q
50                for q in queries}
51     for future in as_completed(futures):
52         try:
53             for d in future.result():
54                 key = (d.metadata.get("source_path"),
55                        d.metadata.get("page"))
56                 if key in seen:
57                     continue
58                 seen.add(key)
59                 gathered.append(d)
60                 if len(gathered) >= 6:
61                     break
62         except Exception:
63             pass
```

Listing 4: Paralelización de queries RAG con `ThreadPoolExecutor`

El mismo patrón se aplicó en `tactics/common.py` (líneas 58–72). Reducción estimada: 60–70% del tiempo de RAG por nodo.

9.2.3.2 P4 — Caché LRU para funciones determinísticas.

Se identificaron cuatro funciones determinísticas (`temperature=0`) cuyos resultados se re-calculaban en cada turno. Se decoraron con `@lru_cache`:

```
18 # Archivo: back/src/graph/nodes/classifier.py (lineas 18-20)
19 @lru_cache(maxsize=256)
20 def _classify_cached(msg: str, qa_opts_str: str) -> tuple:
21     """Cached by (msg, qa_opts_str). temperature=0."""
```

Listing 5: Caché LRU en el clasificador

Las funciones cacheadas incluyen: `_classify_cached` y `_resolve_qa_cached` en `classifier.py`, `_fetch_styles_rag` en `styles/common.py` (línea 28), y `_fetch_tactics_r` en `tactics/common.py` (línea 41).

Adicionalmente, se separaron las instrucciones estáticas del prompt (`SystemMessage`) del contenido dinámico (`HumanMessage`) para habilitar el *prompt caching* del proveedor LLM, que cachea automáticamente prefijos de prompt mayores a 1024 tokens.

9.2.3.3 P5 — Renderizado asíncrono de Graphviz. El renderizado de diagramas usaba `subprocess.run()`, que bloqueaba el event loop de FastAPI durante 5–15 segundos. Se implementó una versión asíncrona con caché compartido:

```
254 # Archivo: back/src/services/diagram_render.py (lineas 254-290)
255 async def render_svg_async(dot_string: str,
256                            engine: str = "dot") -> bytes:
257     key = hash((dot_string, engine))
258     if key in _SVG_CACHE:
259         return _SVG_CACHE[key]
260     try:
261         proc = await asyncio.create_subprocess_exec(
262             engine, "-Tsvg",
263             stdin=asyncio.subprocess.PIPE,
264             stdout=asyncio.subprocess.PIPE,
265             stderr=asyncio.subprocess.PIPE,
266         )
267         stdout, stderr = await asyncio.wait_for(
268             proc.communicate(
269                 input=dot_string.encode("utf-8")),
270             timeout=30,
271         )
272         if proc.returncode == 0:
273             svg_bytes = stdout
274             _SVG_CACHE[key] = svg_bytes
275             return svg_bytes
276     except asyncio.TimeoutError:
277         log.warning("Graphviz async timed out, sync fallback")
278     return render_svg(dot_string, engine=engine)
```

Listing 6: Renderizado asíncrono de Graphviz con caché

El caché `_SVG_CACHE` (línea 196 del mismo archivo) es seguro sin TTL porque Graphviz es puramente funcional: el mismo DOT con el mismo motor siempre produce el mismo SVG.

9.3 Frontend

[TODO: Escribir sección frontend]

9.4 Diseño Anterior del Grafo

El desarrollo de ARCHIA no partió de cero: al inicio del semestre se heredó una primera versión funcional del sistema cuyo núcleo era un único archivo monolítico `graph.py` que contenía la

definición del estado, los nodos, los prompts del sistema, las utilidades auxiliares y el cableado del grafo. Esta sección documenta el estado de esa versión, los problemas identificados y las lecciones que motivaron el rediseño.

9.4.1 Flujo original del sistema

En la versión inicial, el flujo de procesamiento de un mensaje seguía la ruta:

`START` → `boot` → `classifier` → `supervisor` → `router` → *agente especializado* → `unifier` → `END`

Los agentes especializados disponibles eran: `investigator` (consultas RAG), `creator`, `evaluator`, `diagram_agent`, `asr`, `style`, `tactics` y `unifier`. El router aplicaba reglas de "visitar una vez" para evitar ciclos, y los nodos de ASR, estilo y tácticas pasaban directamente al unifier sin regresar al supervisor.

El sistema de diagramas en esta versión generaba código Mermaid que se renderizaba en el frontend con una librería JavaScript, sin validación del lado del servidor.

9.4.2 Diagnóstico: fortalezas y debilidades

Tras una revisión técnica realizada en la reunión de apertura del semestre, se identificaron las siguientes fortalezas y debilidades:

Table 3: Diagnóstico del estado inicial de ARCHIA

Fortalezas	**Debilidades**
Agentes especializados con roles claros	Alto acoplamiento: todo concentrado en `graph.py`
Fundación del RAG implementada con ChromaDB	Documentación deficiente
Arquitectura LangGraph funcional	Respuestas poco específicas al contexto del usuario
Stack tecnológico coherente (FastAPI, React, LangChain)	Sin pruebas unitarias ni end-to-end
Despliegue sencillo	Tiempos de respuesta elevados (hasta 350 s en ciertos prompts)

9.4.3 Problemas críticos de diseño

Más allá del acoplamiento del código, la versión inicial presentaba problemas funcionales que comprometían la utilidad de la herramienta:

1. **No seguía el orden de ADD 3.0.** Los nodos de ASR, estilo y tácticas operaban de forma independiente. No existía una cadena de dependencias que garantizara que el estilo se seleccionara con base en un ASR previo, o que las tácticas fueran coherentes con el estilo elegido.

2. **Respuestas monolíticas.** Cuando el usuario solicitaba varias cosas en un solo prompt (ASR + estilos + tácticas), el sistema generaba una respuesta única que mezclaba todo o simplemente ignoraba parte de la solicitud.

3. **Generación no solicitada de diagramas.** El sistema a veces producía diagramas cuando el usuario no los había pedido, debido a una detección de intención demasiado permisiva.

4. **Pérdida de idioma.** El idioma de respuesta podía cambiar de español a inglés entre turnos, ya que cada nodo detectaba el idioma independientemente.

9.4.4 Feedback del asesor que motivó el rediseño

El asesor del proyecto identificó problemas adicionales de fondo relacionados con la metodología ADD 3.0:

- Las tácticas se generaban demasiado rápido, sin que el usuario hubiera definido primero un estilo arquitectónico.
- El ASR incluía información sobre cómo resolver el problema (tácticas dentro del escenario), cuando debería limitarse a describir el requisito.
- No existía coherencia forzada entre el estilo seleccionado y las tácticas propuestas: el sistema podía recomendar tácticas incompatibles con el estilo.
- Los ASRs se podían inventar a mitad del proceso, rompiendo la secuencia lógica de ADD 3.0.

Estas observaciones motivaron el rediseño completo del grafo que se describe en la Section 9.5.

9.5 Diseño Final del Grafo

El rediseño del grafo abordó los problemas identificados en la Section 9.4 mediante cuatro intervenciones principales: un flujo que respeta el orden de ADD 3.0, soporte para multi-intent, enrutamiento dinámico por atributo de calidad y un mecanismo de trazabilidad de decisiones. Esta subsección describe cada intervención y su implementación.

9.5.1 Flujo actual del grafo

El flujo final del sistema sigue la ruta:

$$\texttt{START} \rightarrow \texttt{boot} \rightarrow \texttt{context_loader} \rightarrow \texttt{classifier} \rightarrow \texttt{supervisor} \rightarrow \texttt{router} \rightarrow \textit{nodo especializado} \rightarrow \texttt{supervisor} \rightarrow \cdots \rightarrow \texttt{unifier} \rightarrow \texttt{END}$$

La diferencia fundamental con la versión anterior es que los nodos especializados regresan al `supervisor` en lugar de ir directamente al `unifier`. Esto permite al supervisor evaluar si quedan nodos pendientes por ejecutar antes de consolidar la respuesta. El cableado se define en `workflow.py`:

```
# Archivo: back/src/graph/workflow.py (lineas 114-139)
builder.add_node("boot", boot_node)
builder.add_node("context_loader", context_loader_node)
builder.add_edge(START, "boot")
builder.add_edge("boot", "context_loader")
builder.add_edge("context_loader", "classifier")
builder.add_edge("classifier", "supervisor")
builder.add_conditional_edges("supervisor", router)
builder.add_edge("investigator", "supervisor")
builder.add_edge("diagram_agent", "supervisor")
builder.add_edge("evaluator", "supervisor")
builder.add_edge("asr", "supervisor")
builder.add_edge("style", "supervisor")
builder.add_edge("tactics", "supervisor")
# ...
builder.add_edge("unifier", END)
graph = builder.compile(checkpointer=sqlite_saver)
```

Listing 7: Cableado del grafo en `workflow.py` (extracto)

9.5.2 Nodos especializados

El sistema cuenta con los siguientes nodos, cada uno implementado como una función que recibe y retorna un `GraphState`:

9.5.2.1 Classifier (`nodes/classifier.py`, líneas 43–110). Detecta el idioma (`es`/`en`), la intención del usuario (`asr`, `style`, `tactics`, `diagram`, etc.) y el atributo de calidad relevante. Utiliza el LLM con `temperature=0` para garantizar determinismo, y el resultado se cachea con `@lru_cache`. La detección de intención combina clasificación LLM con heurísticas de keywords para mayor robustez.

9.5.2.2 Supervisor (`nodes/supervisor.py`, líneas 147–252). Es el orquestador central. Determina qué nodo ejecutar a continuación basándose en la intención detectada, los nodos ya visitados y los nodos pendientes. Incluye el scheduler multi-intent descrito en la Section 9.5.3.

9.5.2.3 ASR (`nodes/asr.py`, líneas 15–151). Genera un Quality Attribute Scenario completo con las seis partes canónicas de ADD 3.0: Source, Stimulus, Environment, Artifact, Response y Response Measure. El prompt fuerza métricas medibles (SLOs como p95, error rate, uptime) y prohibe incluir tácticas o estilos en el escenario. Opcionalmente consulta el RAG para fundamentar el escenario en literatura.

9.5.2.4 Style (`nodes/styles/`). Propone exactamente dos estilos arquitectónicos candidatos, evalúa el impacto de cada uno sobre el ASR activo y recomienda uno con justificación. La salida es un JSON estructurado con `style_1`, `style_2`, `best_style` y `rationale`. El estilo recomendado se almacena en `state["selected_style"]` para uso en nodos posteriores.

9.5.2.5 Tactics (`nodes/tactics/`). Selecciona exactamente tres tácticas de diseño que satisfagan el ASR dado el estilo seleccionado. Cada táctica incluye: nombre, rationale, consecuencias, probabilidad de éxito (0–1) y trazabilidad explícita al ASR. La salida incluye tanto Markdown (para el usuario) como JSON estructurado (para el estado).

9.5.2.6 Diagram (`nodes/diagram.py`). Orquesta la generación de diagramas en tres niveles: overview (5–15 nodos), detailed (todos los nodos) y focused (expansión de un grupo específico). El pipeline es: LLM genera código DOT → parseo a representación intermedia (`DiagramModel`) → simplificación programática si excede 20 nodos → renderizado a SVG vía Graphviz.

9.5.2.7 Investigator (`nodes/investigator.py`). Realiza consultas RAG sobre literatura de arquitectura. Se invoca cuando el usuario hace preguntas teóricas o cuando otro nodo necesita fundamentación bibliográfica.

9.5.2.8 Evaluator (`nodes/evaluator.py`). Analiza y critica ASRs o propuestas arquitectónicas. Evalúa corrección teórica, viabilidad práctica y alineación con los requisitos del usuario.

9.5.2.9 Unifier (`nodes/unifier.py`). Consolida las respuestas de todos los nodos ejecutados en el turno en un mensaje final coherente, incluyendo fuentes RAG y sugerencias de seguimiento.

9.5.3 Soporte multi-intent

Para resolver el problema de solicitudes compuestas (por ejemplo, "genera un ASR, recomienda estilos y selecciona tácticas"), se implementó un scheduler multi-intent en el supervisor. Se añadieron tres campos al `GraphState`:

```
# Archivo: back/src/graph/state.py (lineas 132-135)
requested_nodes: list   # nodos solicitados en este turno
pending_nodes: list     # nodos aun por ejecutar
completed_nodes: list   # nodos ya ejecutados
```

Listing 8: Campos de planificación multi-intent en `GraphState`

La función `_infer_requested_nodes()` en `supervisor.py` (líneas 61–121) analiza el mensaje del usuario y construye un plan de ejecución ordenado. El supervisor respeta las dependencias: si se piden tácticas pero no existe un ASR, el sistema genera primero el ASR, luego ejecuta tácticas. Tras cada nodo, el supervisor verifica si quedan nodos pendientes antes de derivar al unifier.

9.5.4 Enrutamiento dinámico por atributo de calidad

Para permitir lógica diferenciada por atributo de calidad sin condicionales dispersos, se implementó un registro central de atributos y nodos especializados por QA.

El módulo `qa_registry.py` (líneas 57–146) expone funciones como `supported_qas()`, `normalize_qa()` y `style_node_name_for_qa()`. Los nodos se registran dinámicamente en `workflow.py`:

```
# Archivo: back/src/graph/workflow.py (lineas 104-111)
for qa_id in _SUPPORTED_QAS:
    style_name = style_node_name_for_qa(qa_id)
    tactics_name = tactics_node_name_for_qa(qa_id)
    if style_name != "style":
        builder.add_node(style_name,
                         make_style_qa_node(qa_id))
    if tactics_name != "tactics":
        builder.add_node(tactics_name,
                         make_tactics_qa_node(qa_id))
```

Listing 9: Registro dinámico de nodos por atributo de calidad

El router (líneas 47–86 de `workflow.py`) traduce la intención lógica (`style` o `tactics`) al nodo especializado correspondiente (`style_latencia`, `tactics_escalabilidad`, etc.). Si no existe un nodo específico para el atributo detectado, el sistema utiliza el nodo general como fallback. Este diseño permite agregar soporte para un nuevo atributo de calidad editando únicamente `config/indices.json`, sin modificar código.

9.5.5 Design Ledger: trazabilidad de decisiones

El problema más profundo de la versión anterior era la falta de trazabilidad entre decisiones: toda la memoria de una sesión vivía en `arch_flow`, un blob JSON plano donde cada nueva decisión sobreescribía la anterior. No existía vínculo estructural entre el ASR, el estilo que lo satisfacía y las tácticas derivadas.

Se implementó el **Design Ledger**, un árbol de decisiones inmutables donde cada decisión referencia explícitamente a la que la originó. La cadena de trazabilidad es:

ASR → **Estilo** (`traces_to` ASR) → **Táctica** (`parents`: estilo + ASR) → **Diagrama**

El Ledger se implementa en el paquete `ledger/` con cuatro módulos:

```
43 # Archivo: back/src/ledger/types.py (lineas 43-59)
44 class Decision(TypedDict):
45     id: str
46     kind: str           # "asr", "style", "tactics", "diagram"
47     phase: str
48     iteration: int
49     qa: str             # atributo de calidad
50     parents: list[DecisionRef]  # referencias a decisiones padre
51     payload: dict[str, Any]
52     rationale: str
53     sources: list[dict]
54     status: str         # "active", "superseded", "rejected"
55     superseded_by: Optional[str]
56     rejection_reason: Optional[str]
57     created_at: str
58     created_by_node: str
```

Listing 10: Estructura `Decision` del Design Ledger

El mecanismo opera en tres capas: (1) el Ledger mismo almacena las decisiones con sus referencias; (2) el `context_loader` rehidrata el Ledger al inicio de cada turno y traduce las decisiones activas a los campos escalares que los nodos necesitan (`current_asr`, `selected_style`, etc.); y (3) el *dossier binding* inyecta el ASR activo directamente en los prompts de estilo y tácticas, forzando que el campo `traces_to_asr` sea obligatorio en cada táctica.

9.5.6 ASR Write-back y Supersession

Para resolver el problema de que el sistema repetía ASRs descartados, se implementó un mecanismo de *supersession*: cuando el arquitecto pide refinar un ASR, la función `append_decision(kind="as` marca el ASR anterior con `status: superseded` y almacena una referencia al nuevo. El historial es inmutable y monotónico.

Antes de cada generación de ASR, se inyecta un bloque `PRIOR ASR HISTORY` en el prompt (máximo 1500 caracteres) que lista los ASRs previamente descartados. Esto permite al LLM evitar regenerar opciones ya rechazadas. En sesiones nuevas, donde no existe historial, el prompt permanece idéntico al de la versión anterior, garantizando compatibilidad.

10 Descripción de la Herramienta

[TODO: Escribir descripción de la herramienta con walkthrough]

11 Resultados

[TODO: Escribir resultados]

12 Pruebas

[TODO: Escribir pruebas]

13 Conclusiones

[TODO: Escribir conclusiones]

14 Trabajo Futuro

A lo largo del desarrollo de ARCHIA se identificaron líneas de mejora que exceden el alcance de este proyecto de grado pero que constituyen oportunidades valiosas para iteraciones futuras. Se organizan en tres categorías: optimizaciones de rendimiento pendientes, mejoras metodológicas y extensiones funcionales.

Optimizaciones de rendimiento pendientes

De los siete cuellos de botella identificados en la Section 9.2.3, tres permanecen sin resolver:

La **paralelización de llamadas LLM** (P1) representa el 60–70% de la latencia total del sistema. Actualmente, cuando el supervisor determina que deben ejecutarse múltiples nodos (por ejemplo, ASR seguido de estilo y tácticas), cada llamada al LLM se ejecuta secuencialmente. Paralelizar nodos independientes mediante `asyncio.gather` o un ejecutor de hilos reduciría significativamente el tiempo total de respuesta.

La implementación de **streaming al cliente** (P2) permitiría que el usuario vea la respuesta a medida que se genera, reduciendo la latencia percibida. Actualmente, `graph.invoke()` bloquea hasta que todos los nodos completan su ejecución. LangGraph soporta streaming nativo mediante `graph.astream()`, pero integrarlo requiere modificar tanto el backend (endpoints SSE o WebSocket) como el frontend (renderizado incremental).

La limitación a **un solo worker de Uvicorn** (P6) implica que una segunda petición queda en cola mientras se procesa la primera, con esperas de 60–80 segundos. Migrar a Gunicorn con múltiples workers o a un modelo de procesamiento basado en colas permitiría atender solicitudes concurrentes.

Mejoras metodológicas

Los arquitectos de Pragma y el asesor del proyecto identificaron que ARCHIA no implementa una **fase de intake guiado**: el sistema permite al usuario solicitar ASRs sin validar que haya proporcionado suficiente contexto sobre el sistema a diseñar. Implementar *guards* que exijan información mínima antes de la primera decisión de diseño (por ejemplo, dominio del sistema, stakeholders, restricciones conocidas) mejoraría sustancialmente la calidad de los artefactos generados.

ARCHIA cubre los pasos 1–4 de ADD 3.0. Extender el soporte a las **iteraciones completas** del método — incluyendo instanciación de elementos arquitectónicos (paso 5), definición de interfaces (paso 6) y verificación/refinamiento (paso 7) — convertiría a la herramienta en un asistente de ciclo completo.

La **validación automática de ASRs** permitiría verificar que los escenarios generados cumplan con la estructura de seis partes y contengan métricas medibles sin intervención del arquitecto, sirviendo como un mecanismo de control de calidad interno antes de presentar el resultado al usuario.

Extensiones funcionales

El backend multi-tenant desarrollado en NestJS (`archIABack-Negocio`) implementa gestión de organizaciones, proyectos, usuarios y preferencias, pero no está integrado con el backend core de agentes. Completar esta **integración como plataforma SaaS** habilitaría la gestión de múltiples proyectos por usuario, con contexto técnico persistente y preferencias personalizadas de explicación.

La implementación de un **sistema de proyectos** con carga de documentación técnica permitiría al arquitecto subir carpetas completas de documentos de un proyecto, proporcionando al sistema un contexto mucho más rico para la generación de ASRs y la selección de tácticas.

Finalmente, la **evaluación a mayor escala** con más arquitectos, más dominios de aplicación y métricas cuantitativas formales fortalecería la evidencia sobre la utilidad del sistema y permitiría identificar patrones de uso y áreas de mejora con mayor rigor estadístico.

A Anexo A: System Prompts Principales

[TODO: Incluir system prompts]

B Anexo B: Ejemplo Completo de Ejecución

[TODO: Incluir ejemplo completo]

C Anexo C: Documentación Técnica Complementaria

[TODO: Incluir documentación técnica]

Referencias

[TODO: Migrar referencias a archivo .bib y usar biblatex]